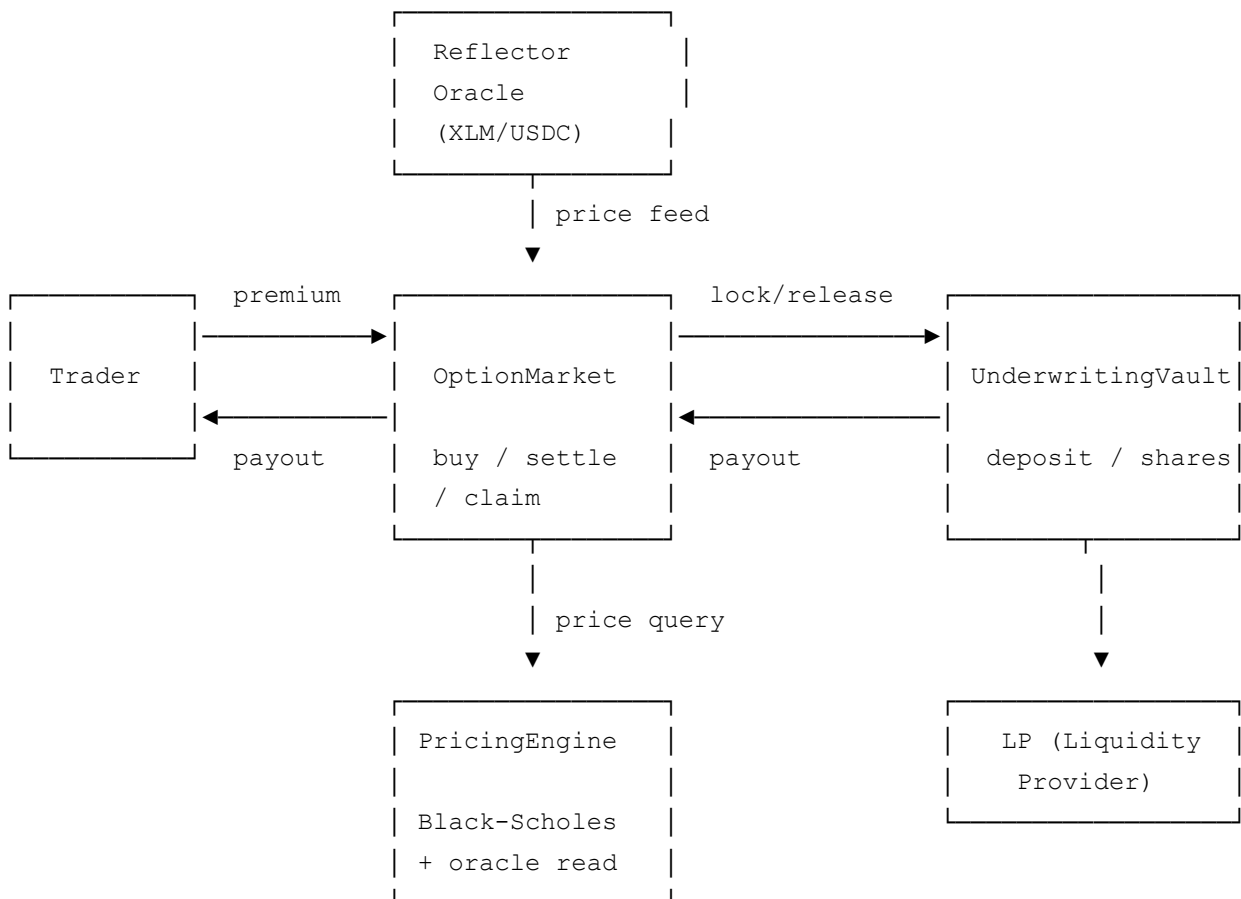


Strix Protocol — Architecture Document

Version: 1.0 **Date:** March 2026

1. Overview

Strix is a peer-to-pool European-style options protocol on Soroban (Stellar). Three smart contracts work together to enable trustless options trading:



2. Contract Relationships

PricingEngine (stateless math + oracle reader)

|

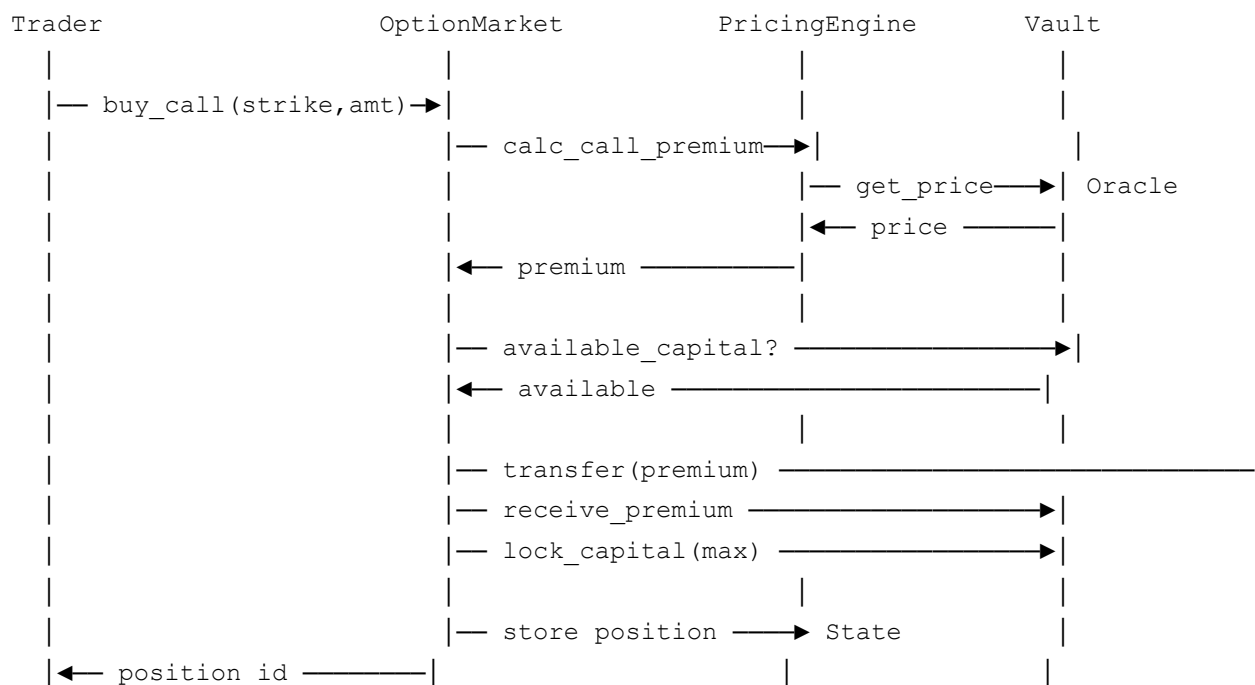


Trust boundaries:

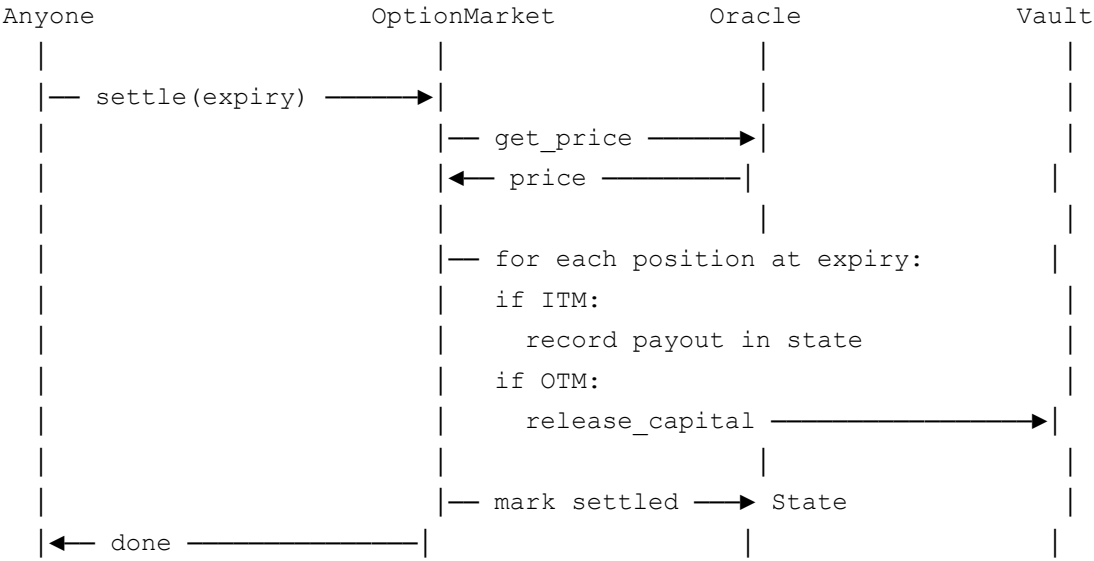
- **UnderwritingVault only accepts** `lock_capital`, `release_capital`, `receive_premium`, and `pay_settlement` **calls from the authorized OptionMarket address**
 - **OptionMarket only reads from PricingEngine (no state modification)**
 - **PricingEngine only reads from Reflector Oracle (no state modification)**
 - **Admin functions on all contracts require** `admin.require_auth()`
-

3. Core Flows

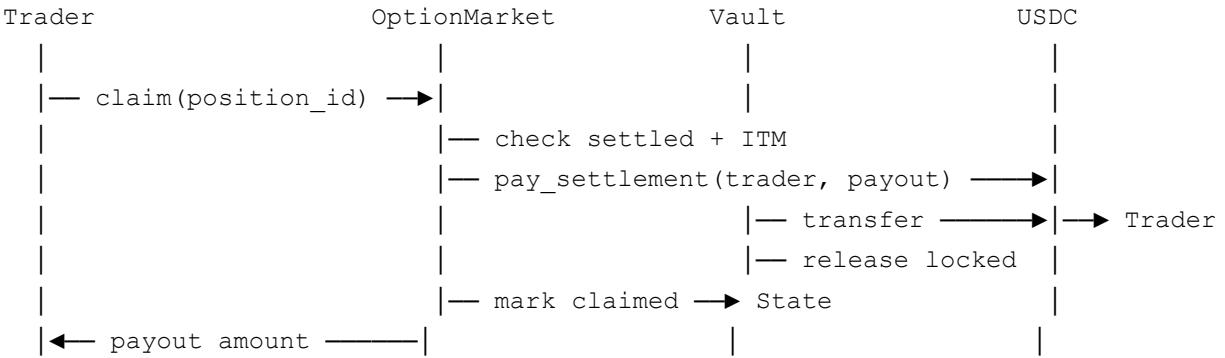
3.1 Option Purchase Flow



3.2 Settlement Flow



3.3 Claim Flow



4. Pricing Architecture

4.1 Black-Scholes Pipeline

Inputs:

- `S` = spot price $\leftarrow$ Reflector Oracle (XLM/USDC)
- `K` = strike price $\leftarrow$ User selection
- `T` = time to expiry $\leftarrow$ $(\text{expiry_timestamp} - \text{current_timestamp}) / \text{SECONDS_PER_YE}$
- `$\sigma$` = implied vol $\leftarrow$ Admin-set (80% MVP)
- `r` = risk-free rate $\leftarrow$ 0 (MVP simplification)

Computation:

```
d1 = [ln(S/K) + (σ²/2)T] / (σ√T)      // r=0 simplifies this
d2 = d1 - σ√T
N(d1), N(d2) via Abramowitz & Stegun rational approximation
Call = S·N(d1) - K·N(d2)               // r=0, no discounting
Put  = K·N(-d2) - S·N(-d1)            // put-call parity
```

Output:

```
Premium = BS_value × (1 + spread) × num_contracts × contract_size
```

4.2 Fixed-Point Arithmetic

All prices and amounts use **i128 with 7 decimal places** (matching USDC precision):

```
1.0000000 USDC = 10_000_000 (i128)
0.1200000 USDC = 1_200_000  (XLM price ~$0.12)
```

Internal math uses 14 decimal places for intermediate calculations,
then rounds down to 7 for final output.

5. Collateral Model

5.1 Capital Reservation

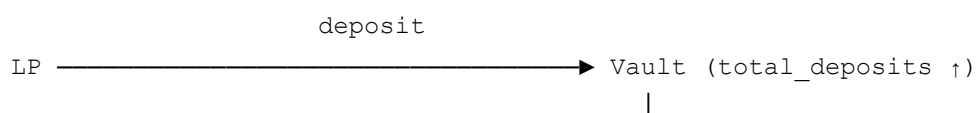
For every option sold, the vault locks sufficient capital to cover the maximum possible payout:

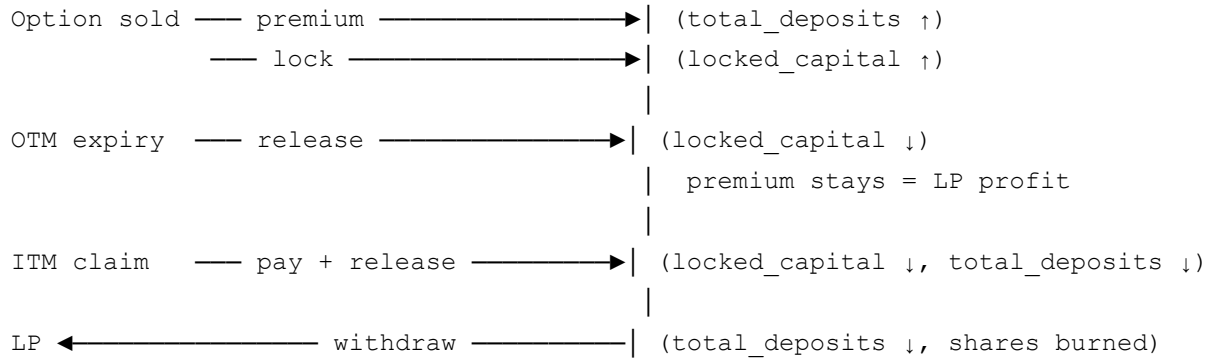
```
Call option lock = contracts × contract_size × strike_price
(Caps maximum loss. In theory calls have unlimited upside,
but capping at strike_price means ~100% price move coverage.
This is a conservative simplification for MVP.)
```

```
Put option lock = contracts × contract_size × strike_price
(Maximum put value = strike price, when underlying goes to 0)
```

```
Available capital = total_vault_value - sum(all_locked_capital)
```

5.2 Capital Flow Lifecycle





6. Storage Architecture

6.1 Soroban Storage Tiers

Instance Storage (shared TTL, cheap)

```

MarketConfig { admin, oracle, vault, ... }
VaultConfig { total_shares, total_deposits, .. }
PricingConfig { admin, iv, spread, ... }
  
```

TTL: Extended on every contract call

Cost: Low (shared across all instance data)

Persistent Storage (per-key TTL, medium cost)

```

Position::{id} → Position data
UserPositions::{address} → Vec<position_ids>
UserShares::{address} → share balance
ExpirySettled::{timestamp} → SettlementInfo
ActiveStrikes::{expiry} → Vec<StrikeInfo>
  
```

TTL: 30 days, extended on access

Cost: Per-key, proportional to data size

Archival: Moves off-chain when TTL expires,
restorable via RestoreFootprintOp

Temporary Storage (auto-deleted, cheapest)

PriceCache:: {timestamp} → cached price
TTL: Short (hours)
Cost: Lowest
Deletion: Permanent when TTL expires

7. Security Model

7.1 Access Control

Function	Who can call
deposit	Any user (with USDC balance)
withdraw	LP who has shares
buy_call	Any user (with USDC for premium)
buy_put	Any user (with USDC for premium)
settle	Anyone (permissionless after expiry)
claim	Position owner only
set_iv	Admin only
set_paused	Admin only
create_epoch	Admin only
set_max_tvl	Admin only
lock_capital	OptionMarket contract only
release	OptionMarket contract only
pay_settle	OptionMarket contract only

7.2 Invariants

These conditions must always hold:

1. `locked_capital <= total_deposits`
2. `sum(all position max_payouts where !settled) == locked_capital`
3. `total_shares > 0` whenever `total_deposits > 0`
4. `share_price >= initial_share_price` (only if no ITM options settled)
5. settled positions cannot be settled again (idempotent)
6. claimed positions cannot be claimed again
7. cannot buy options with expiry in the past
8. cannot buy options when paused
9. settlement only after expiry timestamp

8. External Dependencies

Dependency	Purpose	Risk	Mitigation
Reflector Oracle	XLM/USDC price	Downtime, stale data	Staleness check (5 min max)
USDC SAC	Payment token	Standard Stellar asset	Well-tested, SDF-maintained
Freighter	Wallet connection	Browser extension availability	Also support Lobstr wallet
Soroban RPC	Transaction submission	RPC availability	Use SDF public testnet RPC
Vercel	Frontend hosting	Service outage	Static export fallback possible

9. Future Architecture (Post-MVP)

V2 additions:

- └─ Dynamic IV calculation (from oracle price history)
- └─ Multiple underlyings (BTC/USDC, ETH/USDC)
- └─ TWAP oracle for settlement (5-min average)
- └─ Multiple concurrent expiry windows
- └─ Position tokens (SAC-compatible for composability)

V3 additions:

- └─ Delta hedging module (auto-hedge via Aquarius/Soroswap)
- └─ Structured product vaults (covered call, put-selling)
- └─ RFQ system for professional market makers
- └─ Integration with Blend (options as collateral)